

Algorithmes et Pensée Computationnelle

Programmation de base

Le but de cette séance est d'aborder des notions de base en programmation. Au terme de cette séance, vous serez capable de :

- Comprendre les différentes représentations de nombres.
- Définir des variables et comprendre les types de variables existants.
- Utiliser des notions d'algèbre booléenne.
- Ecrire des branchements conditionnels.

Les langages qui seront utilisés pour cette séance sont Java et Python. Assurez-vous d'avoir bien installé IntelliJ. Si vous rencontrez des difficultés, n'hésitez pas à vous référer au guide suivant : [tutoriel d'installation des outils et prise en main de l'environnement de travail](#).

1 Représentation de nombres entiers

Question 1: (🕒 5 minutes) Entiers non signés

Sur 8 bits, convertir $113_{(10)}$ en base binaire.

Question 2: (🕒 5 minutes) Entiers signés négatifs

En utilisant le résultat de la question précédente, convertir sur 8 bits $-113_{(10)}$ en base binaire. Utiliser la représentation **signée** à la diapositive 10 du cours.

Question 3: (🕒 5 minutes) Complément à 1

Écrire le complément à 1 de $-113_{(10)}$ sur 8 bits.

Quelle est la différence entre cette méthode et la précédente ?

Question 4: (🕒 5 minutes) Complément à 2

Quelle est la représentation du complément à 2 (two complement) de $-113_{(10)}$ sur 8 bits et quelle est l'utilité de cette représentation ?

Question 5: (🕒 10 minutes) Floating point

Voici la représentation en binaire d'un nombre à virgule flottante :

signe	exposant	mantisse
0	10110101	010000010000000000000001

Que vaut cette représentation en base 10 ? Utiliser la représentation des floating points (avec un biais de 127). Arrondir les résultats intermédiaires et la valeur finale au 3ème chiffre significatif après la virgule.

2 Typage

Le but de cette partie des travaux pratiques est de comprendre les notions de typage statique et dynamique, ainsi que leur impact sur l'exécution et la gestion des erreurs.

Question 6: (🕒 5 minutes) Les types de variables

Quelles sont les principaux types de variables (donnez en 3), comment les déclareriez-vous en Python et en Java ?

Question 7: (🕒 10 minutes) Typage statique et dynamique

Parmi ces différents programmes, lesquels pourront être exécutés sans problème et lesquels lèveront des erreurs ? Dans le cas où ils génèreraient des erreurs, expliquez la raison de ces erreurs.

Java :

```
1 // Programme 1
2 int variable_1 = 0;
3 int variable_2 = 3;
4 variable_2 = variable_1;
5
6 // Programme 2
7 var variable_1 = 0;
8 var variable_2 = "Hello";
9 variable_2 = variable_1;
10
11 // Programme 3
12 int variable_1 = 0;
13 String variable_2 = "Hello";
14 variable_2 = variable_1;
15
16 // Programme 4
17 var variable_1 = 0;
18 variable_1 = 3.14 ;
19
20 // Programme 5
21 var variable_1 = 0;
22 String variable_2 = "Hello";
23 String variable_3 = "World";
24 variable_2 = variable_3;
25
26 // Programme 6
27 int variable_1 = 0;
28 variable_1 = 3.14 ;
```

Python :

```
1 # Programme 7
2 variable_1 = 0
3 variable_2 = "Hello"
4 variable_2 = variable_1
5
6 # Programme 8
7 variable_1 = 0
8 variable_1 = 3.14
```

⚠ Erreurs fréquentes

Au cas où vous exécuteriez les programmes 2, 4 et 5 et rencontriez l'erreur suivante : `Exception in thread "main" java.lang.UnsupportedClassVersionError: ... has been compiled by a more recent version of the Java Runtime (class file version 54.0), this version of the Java Runtime only recognizes class file versions up to 52.0`, suivez les étapes ci-dessous :

1. Mettez à jour votre JDK en téléchargeant la version adéquate via le lien suivant : <https://www.oracle.com/java/technologies/downloads/>.
2. Une fois votre JDK installé, redémarrez IntelliJ
3. Dans la barre de menus, cliquez sur **⚙ > Edit...**
4. Dans la fenêtre qui apparaîtra (capture ci-dessous), sélectionnez la version du JRE la plus récente.

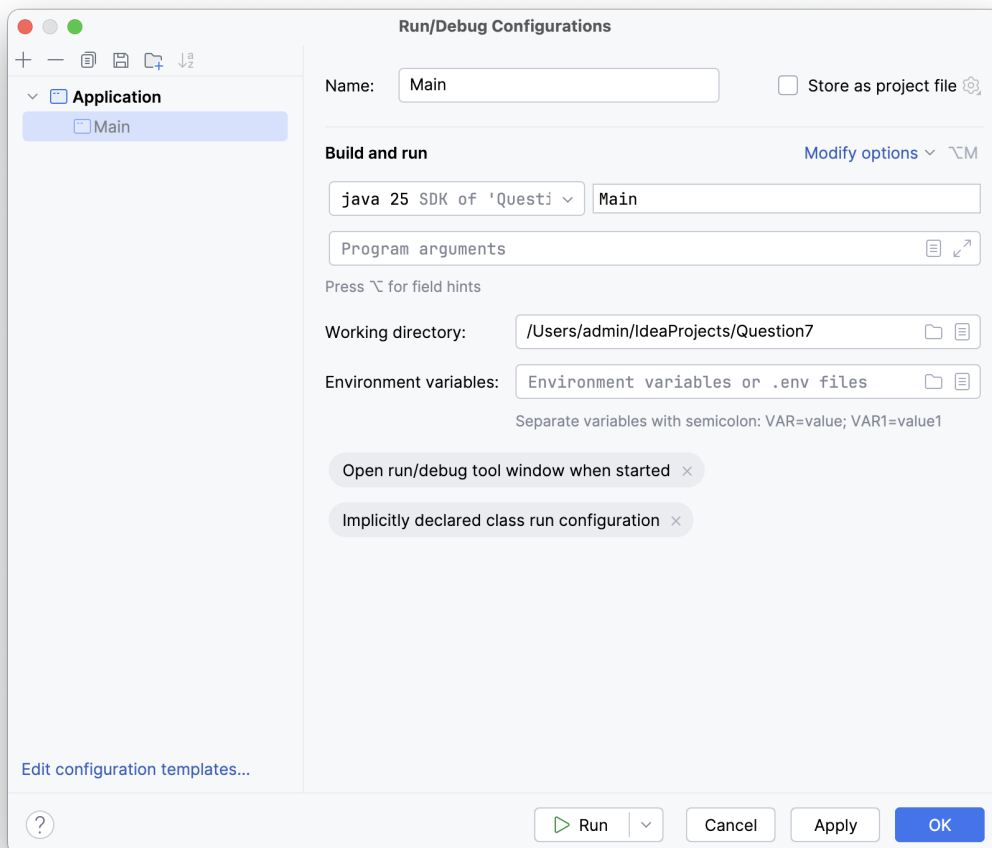


FIGURE 1 – Fenêtre de configuration de l'environnement de développement Java.

3 Opérateurs et conditions Booléennes

Le principe d'une valeur booléenne est qu'elle ne puisse contenir que 2 valeurs possibles, soit `True`, soit `False`. Il est possible de les définir en leur associant une de ces valeurs d'emblée ou de les obtenir en effectuant une opération booléenne comme par exemple une comparaison. Pour ce faire, il faut utiliser des opérateurs booléens. Voici les plus utilisés : `=` (est égal), `!=` (n'est pas égal), `<` (est strictement plus petit), `≤` (est plus petit ou égal), `>` (est strictement plus grand), `≥` (est plus grand ou égal). Si la condition est satisfaite, on obtiendra `True`, si elle ne l'est pas, on obtiendra `False`. L'utilisation de l'opérateur `not` inversera le résultat.

Question 8: (🕒 7 minutes)

1. Démontrez l'équivalence suivante :

$$(\neg p \vee q) \wedge (p \vee \neg q) \Leftrightarrow (p \wedge q) \vee (\neg p \wedge \neg q)$$

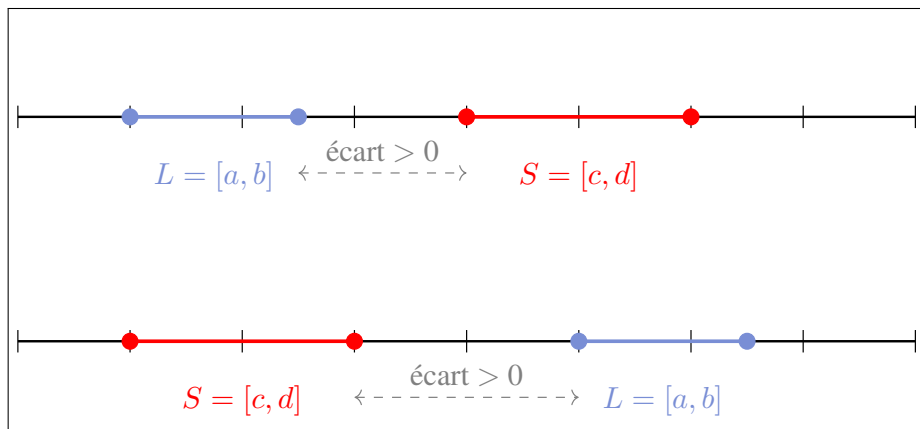
2. Re-faire cet exercice en utilisant les tables de vérité.

Question 9: (🕒 2 minutes)

⚠ Attention

Dans l'exercice suivant, vous pouvez supposer toujours que $b > a$ et $d > c$ pour les intervalles $L = [a, b]$ et $S = [c, d]$.

1. À l'aide de la figure ci-dessous, écrivez la condition booléenne en fonction de a, b, c, d pour que les deux intervalles ne se recouvrent pas.
2. Re-écrire la négation de cette condition à l'aide de la loi DeMorgan (diapositive 23 du cours).



Dans les exercices suivants, vous devrez anticiper la valeur que la console va vous donner (résultat du print).

Question 10: (🕒 5 minutes) Qu'affiche le programme suivant ?

```
1 a = 3
2 b = 2
3 c = 6
4 d = c >= b
5 print(a==b or d)
6 print(a==b and d)
7 print(a==b or a==c or False or d)
8 print(a<c and not (4*b==8 and not d))
```

Question 11: (🕒 5 minutes)

1. Qu'affiche le programme suivant ?

```

1 a = True
2 b = False
3 c = True
4 if a or (b and not c):
5     print("output 1")
6 if b!= c :
7     print("output 2")
8 if a or (not b != (c and a)):
9     print("output 3")
10 else:
11     print("output 4")

```

2. Qu'affiche le programme si l'on change la ligne 6 à `elif b != c`?

4 Match (Python) et Switch (Java)

L'instruction `match` en Python et `switch` en Java a pour objectif de simplifier la logique conditionnelle en sélectionnant un bloc de code à exécuter parmi plusieurs options. Cette structure compare la valeur d'une variable ou d'une expression à une série de cas (case) prédéfinis. Lorsqu'une correspondance est trouvée, le code associé à ce cas est exécuté, offrant ainsi une alternative plus lisible et organisée qu'une longue chaîne de `if-elif-else`.

Consultez [cette mémoire disponible sur Moodle](#) pour comprendre le fonctionnement d'un `switch` en Java.

Question 12: (🕒 5 minutes) Écrire le code Java qui demande à l'utilisatrice d'entrer l'étape d'exécution d'une instruction et affiche les étapes qui restent à exécuter (y compris l'étape courante). Si l'utilisatrice entre une étape invalide le code affiche L'étape n'est pas valide.

Exemple

Si l'utilisatrice entre `decode`, le programme va afficher :

L'étape `decode` s'exécute ou va être exécutée.
 L'étape `execute` s'exécute ou va être exécutée.
 L'étape `store` s'exécute ou va être exécutée.